R Cheatsheet

Teaching Week 01 to 12 | The Python sheet carries the same sections in the same order.

The Python cheatsheet carries the same nine sections with the same entries in the same positions, so a student who knows one language can find the other call by reading across. Every example works on `heart.csv`, the table Class 01 opens with, through its `Age`, `MaxHR`, `Cholesterol`, and `HeartDisease` columns. The exceptions are the few lines whose comment names a stand-in (`lookup`, `num_obs`, `num_pred`), which need an object of your own. Load a package once at the top of your script rather than beside every call that needs it.

1 Reading a Table

```r
## Read a comma-separated file into a data frame.
heart <- read.csv("heart.csv", stringsAsFactors = TRUE)

## State the separator and the decimal mark when they are not the defaults.
heart <- read.csv("heart.csv", sep = ",", dec = ".")

## Read the first rows only, while you check that the file parses.
read.csv("heart.csv", nrows = 20)

## Treat the strings a source writes for "no value" as missing.
read.csv("heart.csv", na.strings = c("", "NA", "-99"))
```

2 Inspecting It

Confirm the shape, the names, and the type of every column before computing a single statistic.

```r
dim(heart)                 # rows and columns
names(heart)               # column names
head(heart, 5)             # the first five rows
str(heart)                 # type and first values of every column
summary(heart)             # a per-column summary of the whole table
table(heart$HeartDisease)  # a count for every level of one column
```

3 Summary Statistics

```r
mean(heart$Age)                      # arithmetic mean
median(heart$Age)                    # middle value
sd(heart$Age)                        # standard deviation, divisor n - 1
var(heart$Age)                       # variance, the square of the above
range(heart$Age)                     # smallest and largest value
quantile(heart$Age, c(0.25, 0.75))   # first and third quartile
IQR(heart$Age)                       # interquartile range, Q3 - Q1
cor(heart[c("Age", "MaxHR")])        # correlation matrix of two columns
```

4 Missing Values

Count the missing values before dropping or filling any, and set the NA handling of every statistic explicitly.

```r
## heart.csv writes an unmeasured cholesterol as 0, so recode it before counting.
heart$Cholesterol[heart$Cholesterol == 0] <- NA
sum(is.na(heart$Cholesterol))    # how many values of one column are missing
colSums(is.na(heart))            # the same count for every column
which(is.na(heart$Cholesterol))  # the positions of the missing values
complete.cases(heart)            # TRUE where a row has no missing value
clean <- na.omit(heart)          # drop every incomplete row
heart$Cholesterol[is.na(heart$Cholesterol)] <-
  median(heart$Cholesterol, na.rm = TRUE)   # fill with the median instead
mean(heart$Age, na.rm = TRUE)    # a statistic skips NA only when told to
```

5 Plotting

The base call is for a quick look at your screen. The grammar call is for the figure the report cites.

```r
library(ggplot2)

hist(heart$Age)                  # base histogram, quick look
boxplot(heart$Age)               # base boxplot, quick look
plot(heart$Age, heart$MaxHR)     # base scatter plot, quick look

ggplot(heart, aes(Age)) +
  geom_histogram(binwidth = 3)   # histogram with a bin width you chose

ggplot(heart, aes(Age, MaxHR, colour = factor(HeartDisease))) +
  geom_point(alpha = 0.6) +      # scatter plot, coloured by the outcome
  labs(x = "Age", y = "Maximum heart rate") +
  theme_minimal()

ggsave("results/age-maxhr.png")  # write the last plot to a file
```

6 Transforming

```r
library(dplyr)

heart |>
  filter(Age >= 40) |>                      # keep the rows that qualify
  select(Age, MaxHR, HeartDisease) |>       # keep the columns you use
  mutate(band = cut(Age, breaks = 4)) |>    # add a derived column
  arrange(desc(MaxHR)) |>                   # sort, largest first
  group_by(HeartDisease) |>                 # form the groups
  summarise(n = n(), mean_age = mean(Age))  # one row per group

scale(heart[c("Age", "MaxHR")])   # centre each column and divide by its sd
factor(heart$HeartDisease)        # read a stored code as a category
## lookup stands in for a second table that shares an id column with heart.
left_join(heart, lookup, by = "id")   # add the columns of a second table
```

7 A Decision Tree

The formula names the outcome on the left and the predictors on the right.

```r
library(rpart)
library(rpart.plot)

fit <- rpart(
  factor(HeartDisease) ~ Age + MaxHR,   # outcome ~ predictors
  data   = heart,
  method = "class",                     # "anova" predicts a number instead
  control = rpart.control(cp = 0.01, minsplit = 20, maxdepth = 2)
)

rpart.plot(fit)                       # draw the fitted tree
printcp(fit)                          # complexity table, one row per split
## cp is alpha / R(root) on misclassification risk, not Python's ccp_alpha,
## so the two values differ. Pick a cp between two rows of the table.
fit <- prune(fit, cp = 0.1)           # cut the tree back to a complexity
predict(fit, heart, type = "class")   # the predicted class of every row
```

8 K-Means Clustering

Scale first, because the algorithm measures distance and an unscaled column decides the answer on its own. Fix the seed so the run repeats.

```r
library(cluster)

Z <- scale(heart[c("Age", "MaxHR")])   # comparable units before any distance
set.seed(3141)
km <- kmeans(Z, centers = 3, nstart = 25, iter.max = 100)

km$cluster                        # the cluster label of every row
km$centers                        # the k centres, in scaled units
km$tot.withinss                   # total within-cluster sum of squares
s <- silhouette(km$cluster, dist(Z))   # the silhouette width of every row
mean(s[, 3])                      # the average silhouette width
```

9 Evaluating a Model

Split the data, score the rows the model has not seen, and report the metric the outcome type calls for.

```r
library(caret)

set.seed(3141)
idx   <- sample(nrow(heart), floor(0.7 * nrow(heart)))
train <- heart[idx, ]                # 70 percent to fit on
test  <- heart[-idx, ]               # the rest to be scored on

fit  <- rpart(factor(HeartDisease) ~ Age + MaxHR,
              data = train, method = "class")
pred <- predict(fit, test, type = "class")
obs  <- factor(test$HeartDisease)

table(obs, pred)             # confusion matrix, observed against predicted
mean(pred == obs)            # accuracy, the share the model got right
confusionMatrix(pred, obs, positive = "1")  # all three, with 1 as positive

train(factor(HeartDisease) ~ Age + MaxHR, data = heart, method = "rpart",
      trControl = trainControl(method = "cv", number = 10))   # 10-fold CV

## num_obs, num_pred stand in for observed and predicted values of a number.
postResample(pred = num_pred, obs = num_obs)   # RMSE, R squared, and MAE
R2(num_pred, num_obs, form = "traditional")    # R squared as 1 - SSres/SStot
```